



Royal Square Financial

Platform Architecture & Technical Planning Document

DOCUMENT TYPE

Architecture Spec

VERSION

1.0 — 2026

CLASSIFICATION

Confidential

ROYAL SQUARE FINANCIAL (PTY) LTD · FSP PLATFORM · SOUTH AFRICA

Table of Contents

Complete platform architecture and planning reference

1 Executive Summary & Vision

Business Context & Problem Statement	Platform Vision
Key Platform Statistics	Regulatory Context

2 Multi-Tenant Architecture

Tenant Isolation Model	Row-Level Security (RLS)
Plug-and-Play Module System	Tenant Onboarding Flow

3 The Four Portals

Client Portal	Admin Portal (Advisers & Staff)
Super Admin Portal	Insurer Onboarding Portal

4 Complete User Flows

Guest Lead Capture & Magic Link	Client Onboarding Wizard
Application → Quote → Inception	Claims Flow

5 Identity & Access Management (IAM)

Permission Matrix	Groups & Individual Overrides
Security Policies	Delegation Rules

6 Database Schema

Core Tenant & IAM Tables	Client CRM Tables
Policy & Claims Tables	Compliance & Audit Tables

7 AI Systems

Recommendation Engine	RAG Architecture
Client AI Assistant	JSON Styling System

8 Integration Gateway

Multi-Insurer Abstraction Layer	Supported Providers
---------------------------------	---------------------

9 Notifications, Search & Accessibility

Notification Channels

In-App Search Engine

Accessibility & Language System

Colour Blind & Screen Reader Support

10 Infrastructure & Compliance

AWS af-south-1 Architecture

POPIA / FAIS / FICA Compliance

Technology Stack

Implementation Roadmap

THE CORE PROBLEM BEING SOLVED

Royal Square Financial currently operates as a traditional independent insurance broker — managing client portfolios, insurer relationships, compliance obligations, and policy administration largely through manual processes: paper files, spreadsheets, email threads, and telephone follow-ups. This creates risk (missed renewals, compliance gaps, lost documents), limits scale (adviser capacity is capped by manual workload), and delivers a poor client experience (no real-time visibility into their own financial position).

Platform Vision

RoyalSync is a **multi-tenant SaaS platform** that digitises and automates the complete lifecycle of independent insurance brokerage operations. Royal Square Financial is Tenant #1 — the platform is architected from day one to be licensed to other independent brokers across South Africa and, ultimately, across Africa and globally.

4

SEPARATE PORTALS

11

BOUNDED DOMAINS

13LANGUAGES
SUPPORTED**100%**

TENANT ISOLATED

What the Platform Eliminates

Manual Operations Removed

- Paper-based client files and KYC folders
- Email-based quote requests to insurers
- Spreadsheet tracking of policy renewals
- Manual commission reconciliation
- PDF forms for every client interaction
- Phone-based claim status follow-ups
- Manual reminder calendars for KYC renewals
- Ad-hoc document distribution

What Replaces It

- Digital client profiles with full audit history
- Integration gateway auto-routing to insurers
- Automated renewal alerts with SLA tracking
- Real-time commission ledger
- Guided wizard flows with digital signatures
- Real-time claim tracking in the client app
- Automated KYC expiry management
- Template-driven document workflows

Regulatory Framework

POPIA

Protection of Personal Information Act — governs all personal data. Requires consent management, right to erasure (pseudonymisation), data breach

FAIS

Financial Advisory & Intermediary Services Act — governs advice and intermediary services. Requires Record of Advice (ROA), commission disclosure, FNA

FICA

Financial Intelligence Centre Act — anti-money laundering and KYC obligations. Requires identity verification, PEP/sanctions

notification within 72 hours, and data residency in South Africa.

(Financial Needs Analysis), 5-year document retention.

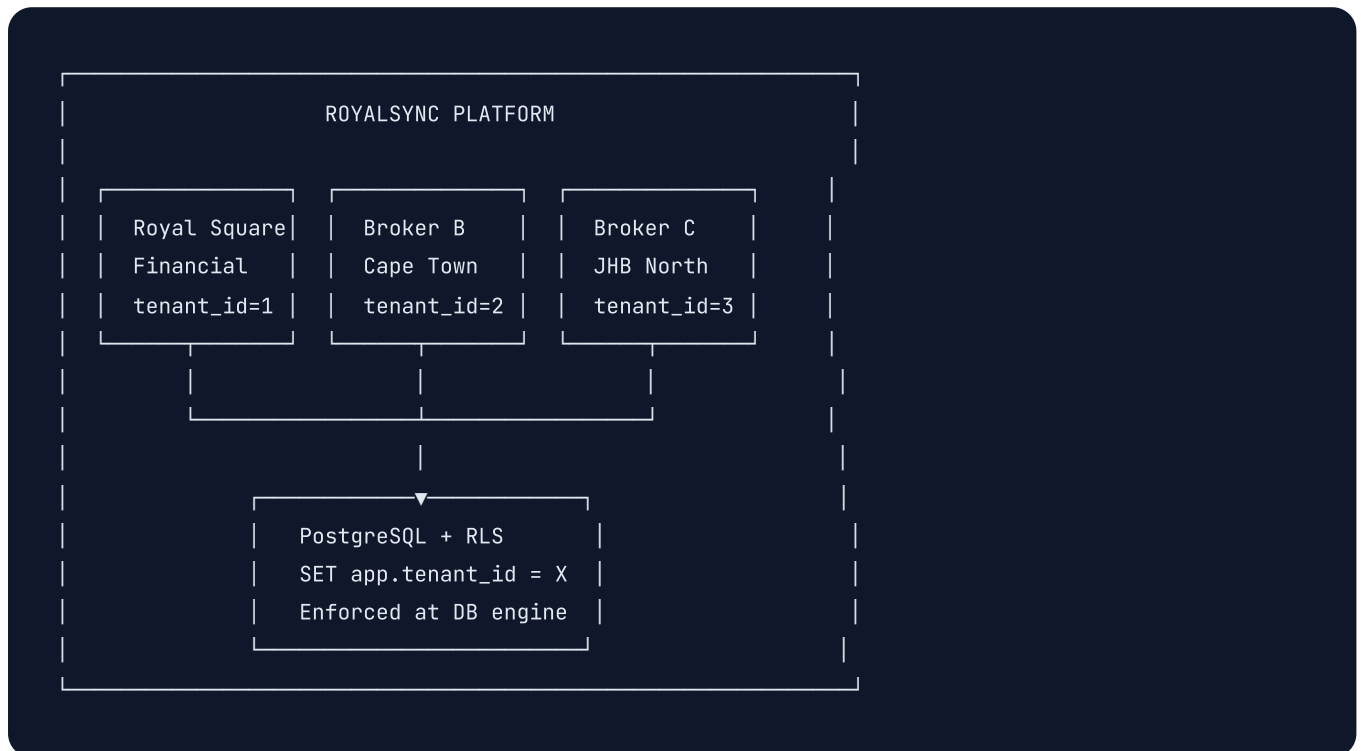
screening, risk scoring, and suspicious activity reporting.

PLATFORM DIFFERENTIATION

Most broker management systems in South Africa are legacy desktop applications (e.g. Flexi, BrokerPoint) that were not designed for modern web, mobile, or multi-tenancy. RoyalSync is cloud-native, mobile-first, AI-assisted, and built to be licensed — giving Royal Square Financial a second revenue stream as the platform grows.

Tenant Isolation Model

The platform uses a **shared schema, row-level security** model. All tenants share the same PostgreSQL database infrastructure, but every table has a `tenant_id` column and Row-Level Security policies enforce that no query can ever return rows from another tenant — at the database engine level, not the application layer.



Row-Level Security Implementation

```
-- On every table:
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON clients
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

-- Set at the start of every request:
SET app.current_tenant_id = '<tenant_uuid>';
-- This is set by the API middleware from the JWT token.
-- No application code can bypass this – it is enforced at DB level.
```

Plug-and-Play Module System

Each tenant can enable or disable platform modules independently. Modules are rows in the `tenant_modules` table. When a module is disabled for a tenant, its navigation items disappear, its API endpoints return 403, and its data is excluded from all queries.

Module	Description	Default	Config Options
core_crm	Client management, profiles, documents	Always on	—
policy_management	Policies, schedules, endorsements	Always on	—
claims	Claims tracking, third parties, updates	Enabled	Auto-assignment rules
goals	Financial goals, projections, contributions	Enabled	Goal types, projection model
kyc	KYC/AML screening, PEP checks	Enabled	Screening provider, risk threshold
financial_planning	Assets, liabilities, net worth, FNA	Enabled	—
ai_assistant	Client-facing AI assistant (RAG)	Enabled	AI provider, document sources
referrals	Referral tracking, rewards	Disabled	Reward structure, max reward
whatsapp	WhatsApp Business API notifications	Enabled	WABA ID, phone number ID
debicheck	DebiCheck/NAEDO mandate management	Enabled	Sponsoring bank, collection day
market_benchmarks	Premium benchmarking against market	Disabled	Update frequency, data source

Tenant Branding

Each tenant has a branding configuration stored in `tenant_branding`. The client portal, email templates, PDF documents, and SMS messages all render with the tenant's own logo, colours, and contact details — clients never see "RoyalSync".

Visual Identity

- Logo (SVG + PNG variants)
- Primary & secondary brand colours
- Font family selection
- Favicon

Communication

- Email from-name & reply-to
- SMS sender name
- WhatsApp display name
- Custom domain (app.broker.co.za)

Legal & Compliance

- FSP licence number
- FAIS representative number
- Privacy policy URL
- Terms & conditions URL

PORTAL SEPARATION PRINCIPLE

Each portal is a completely separate authentication boundary with its own URL, its own JWT audience claim, its own session store, and its own permission context. A user authenticated in one portal cannot access another portal — even with the same email address — without a separate authentication event.

**Client Portal**

app.royalsquare.co.za

Progressive Web App

Offline-capable

The client-facing experience. Every insured client gets access. Mobile-first PWA with service worker caching for offline use.

Dashboard — Net worth, cover summary, upcoming payments, alerts

Insurance — All active policies, schedules, endorsements, OUTbonus

Claims — Real-time status, documents, third parties, updates

Goals — Financial goals, progress, projections, contributions

Documents — All documents, download, share, upload

AI Insights — RAG-powered assistant for financial questions

Profile — Personal details, bank accounts, dependants

Payments — Payment history, DebiCheck mandates, outstanding

Settings — Language, accessibility, notifications, security

**Admin Portal — Advisers & Staff**

admin.royalsquare.co.za

Internal Staff Only

Used by advisers, case managers, admin staff, and team leads. Permission-gated — each user only sees what their group allows.

Dashboard — My clients, SLA breaches, today's tasks, alerts

Clients — Full CRM, search, profile management, KYC

Applications — Pipeline view, quote management, inception

Policies — Portfolio view, renewals, endorsements, schedules

Claims — Case management, insurer liaison, client updates

Tasks — Assigned tasks, templates, SLA tracking

Templates — Quote requests, letters, document templates

Notifications — Send WhatsApp, email, SMS to clients

Reports — Portfolio, commission, KYC, SLA reports

**Super Admin Portal — Leadership**

superadmin.royalsquare.co.za

Royal Square Leadership

Two-person sign-off

Platform command centre. Full IAM, all tenants (if platform grows), system configuration, compliance oversight, AI management.

IAM — Users, groups, permissions, security policies

Compliance — ROA status, KYC gaps, POPIA, FICA oversight

Analytics — Revenue, portfolio health, adviser performance

Tenants — Manage broker tenants, modules, branding

Insurers — Approved insurer list, integration status

System Config — Templates, notification settings, AI config

Audit Log — Immutable full-platform audit trail

AI Management — Document corpus, model config, conversation logs

Integration Health — Gateway status, dead-letter queue, retry stats



Insurer Onboarding Portal

partners.royalsquare.co.za

External Partners

Insurers use this portal to self-onboard onto the platform, maintain their profile, manage their product library, and receive integration requirements.

Company Profile — FSP details, contact persons, compliance docs

Products — Product catalogue, cover types, terms, pricing guides

Document Requirements — What docs to request from clients per product

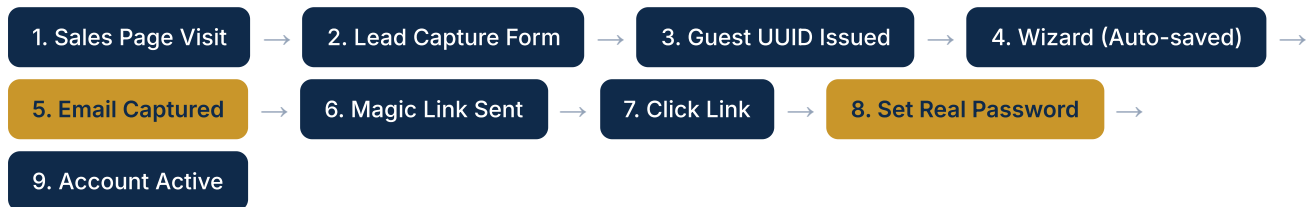
Integration Setup — API keys, SFTP config, email routing

Inbox — New quote requests from Royal Square

Response Centre — Submit quotes, decline reasons, counter-offers

Flow 1 — Guest Lead Capture & Magic Link

A prospect arrives on the Royal Square sales page. They have not registered. All data they enter is preserved server-side so they can resume on any device.



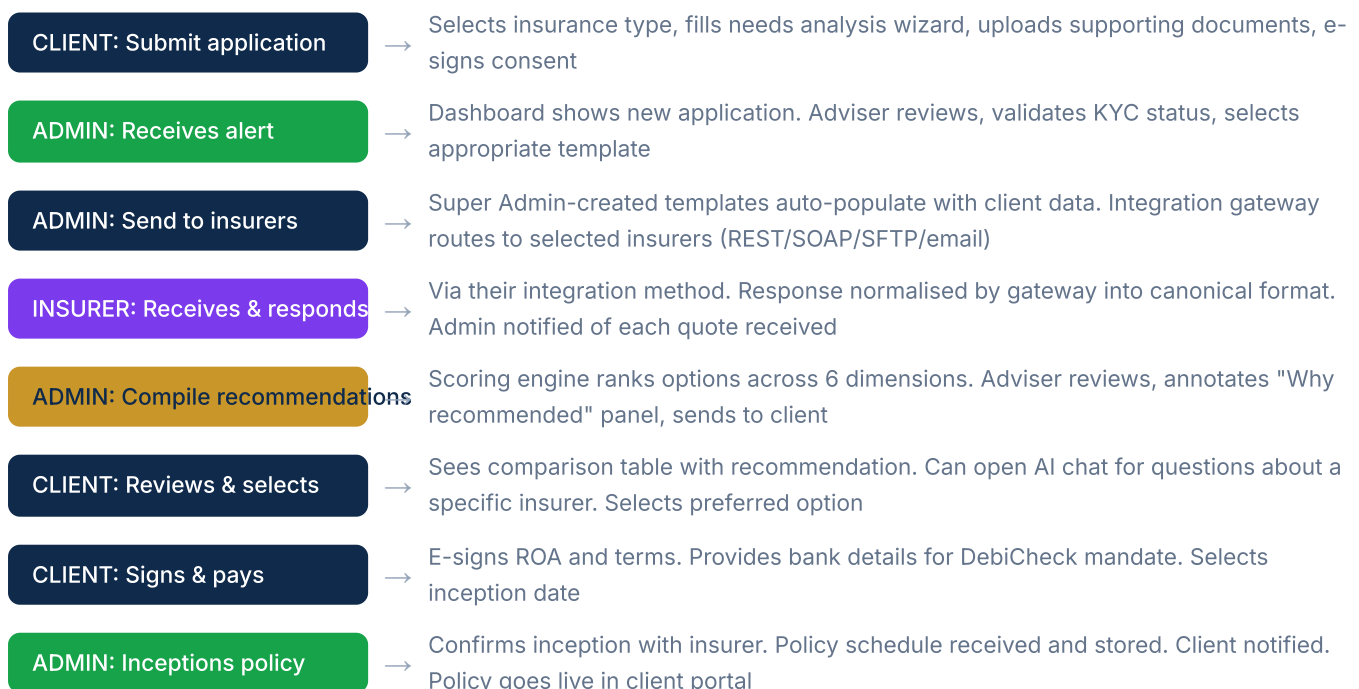
Guest Session (pre-email)

- UUID guest token in browser storage
- All form data saved server-side every 3 seconds
- Guest can close browser and return — data is there
- No account exists yet — just a guest session record
- Guest token expires after 30 days of inactivity

Magic Link Security

- One-time UUID token — invalidated on first click
- 7-day expiry from time of issue
- Second click → redirect to login page with message
- Expired link → form to request a new magic link
- Guest wizard data pre-loaded into new account on activation

Flow 2 — Application → Quote → Inception



Flow 3 — Claims



Assessor assigned



Updates pushed to client



Settlement / repair



Claim closed

OFFLINE & NETWORK RESILIENCE

All wizard flows auto-save every 3 seconds to the server. Uploaded files use chunked upload with resume capability — if the network drops mid-upload, the client selects nothing on return; the upload resumes from the last successful chunk. The client portal is a PWA with a service worker — clients can view their policies, documents, and claim status even when fully offline using cached data. Any actions taken offline (e.g. typing a message) are queued and submitted when connectivity returns.

Permission Matrix — Resource × Action × Scope

Every permission is the intersection of three dimensions. No permission exists outside this model.

Resource	Actions	Scopes
clients	view, create, edit, delete, export	own · branch · tenant · global
policies	view, create, edit, delete, inception, endorse	own · branch · tenant · global
claims	view, create, edit, close, reopen	own · branch · tenant · global
documents	view, upload, download, delete	own · branch · tenant · global
tasks	view, create, assign, complete, delete	own · branch · tenant · global
users	view, create, edit, deactivate, impersonate	branch · tenant · global
reports	view, export	own · branch · tenant · global
templates	view, create, edit, delete, publish	branch · tenant · global
iam	view, create_group, edit_group, assign_permissions	tenant · global
audit_log	view, export	branch · tenant · global
financial_data	view, edit	own · branch · tenant · global
commissions	view, reconcile, export	own · branch · tenant · global

Groups (Roles) — Pre-defined for Royal Square

Sales Agent

- Create & view clients (own scope)
- Create applications
- View own task list
- Send templated messages
- Cannot view other agents' clients
- Cannot view financial data

Adviser

- Full client management (own scope)
- Policy inception & endorsement
- Claims management
- Financial needs analysis
- ROA creation
- Commission view (own)

Case Manager

- Claims management (branch scope)
- View all client files (branch)
- Task assignment
- Insurer communication
- Cannot create policies
- Cannot edit financial data

Branch Manager

- All of the above (branch scope)

Compliance Officer

- View all client files (tenant scope)

Super Admin

- All resources, all actions
- IAM management
- System configuration

- View all commission reports (branch)
- Assign advisers to clients
- Performance reports
- Cannot manage IAM

- View ROA & FNA status (all)
- View KYC status (all)
- View audit log
- Cannot edit any client data

- Sensitive actions require two-person sign-off

Two-Person Sign-Off for Sensitive Operations

CRITICAL PERMISSION — REQUIRES TWO SUPER ADMINS

The following actions require a second super admin to approve before executing: delete or archive a client record, bulk export of client PII, modify another super admin's permissions, deactivate a user with active policies, reset MFA for any user, change bank mandate details without client consent flow, access audit log for a specific user (privacy protection). The first super admin initiates — the system sends an approval request to the second super admin. The action is held pending for 24 hours. Either super admin can cancel.

Individual Permission Overrides

On top of group-level permissions, a super admin can grant or revoke specific permissions for an individual user. Individual overrides are shown as checkboxes alongside each permission in the user's profile. An override does not change the group — it creates a delta record that is merged at permission resolution time.

Security Policies

Policy	Default	Configurable per tenant
MFA requirement	Required for all admin/super admin	Yes — can require for clients too
Session duration	8 hours (admin), 30 days (client remember-me)	Yes
Password policy	Min 12 chars, 1 upper, 1 symbol, no common words	Yes — can strengthen, not weaken
IP allowlist	Off	Yes — can restrict admin portal to office IPs
Login attempt lockout	5 attempts → 15 min lockout	Yes
Concurrent sessions	3 max per user	Yes

DESIGN PRINCIPLES

PostgreSQL 16 with Row-Level Security. Every table has: `tenant_id` UUID NOT NULL , `created_at` TIMESTAMPTZ DEFAULT NOW() , `updated_at` TIMESTAMPTZ , `deleted_at` TIMESTAMPTZ (soft delete — never hard delete due to FAIS retention). Audit log is INSERT-only, enforced by a PostgreSQL trigger that raises an exception on UPDATE or DELETE.

Tenant Management

Column	Type	Notes
id	UUID PK	<i>gen_random_uuid()</i>
name	VARCHAR(255)	<i>Broker display name</i>
slug	VARCHAR(100) UNIQUE	<i>URL prefix — e.g. "royalsquare"</i>
fsp_number	VARCHAR(50)	<i>FSCA FSP licence number</i>
subscription_tier	ENUM	<i>starter / professional / enterprise</i>
data_region	VARCHAR(30)	<i>aws:af-south-1 (POPIA) / aws:eu-west-1</i>
settings	JSONB	<i>Per-tenant configuration overrides</i>

Users & IAM

Column	Type	Notes
id	UUID PK	
tenant_id	UUID FK	→ <i>tenants.id</i>
email	VARCHAR(320)	<i>Unique within tenant</i>
password_hash	TEXT	<i>bcrypt, cost factor 12</i>
user_type	ENUM	<i>client / staff / super_admin</i>
mfa_secret	TEXT ENCRYPTED	<i>TOTP secret, AES-256 at rest</i>
last_login_at	TIMESTAMPTZ	
accessibility_prefs	JSONB	<i>language, colour_mode, text_size, motion, font</i>

Client CRM

Column	Type	Notes
id	UUID PK	
tenant_id	UUID FK	<i>RLS enforced</i>
user_id	UUID FK	→ <i>users.id</i> (the login account)
id_number	VARCHAR(20) ENCRYPTED	<i>SA ID or passport — AES-256</i>

Column	Type	Notes
first_name, last_name	VARCHAR(100)	
date_of_birth	DATE	
mobile	VARCHAR(20)	E.164 format
risk_profile	ENUM	low / medium / high
kyc_status	ENUM	pending / verified / expired / rejected
kyc_verified_at	TIMESTAMP TZ	KYC expires 3 years after this
referral_code	VARCHAR(20) UNIQUE	Their shareable referral code
referred_by	UUID FK	→ clients.id (NULL if not referred)
assigned_adviser_id	UUID FK	→ users.id

Policies

Column	Type	Notes
id	UUID PK	
tenant_id	UUID FK	
client_id	UUID FK	→ clients.id
provider_id	UUID FK	→ product_providers.id
policy_number	VARCHAR(100)	Insurer's policy number
product_type	ENUM	motor / home_building / home_contents / life / medical / business / travel / funeral / gap / income_protection
status	ENUM	active / lapsed / cancelled / not_taken_up / expired
premium_amount	NUMERIC(12,2)	Current monthly premium
premium_model	ENUM	fixed / annual_escalation / decreasing / pay_per_km / behaviour_based
inception_date	DATE	
renewal_date	DATE	Triggers renewal alert 60 days before
outbonus_streak_years	INTEGER DEFAULT 0	OUTsurance claim-free streak
commission_rate	NUMERIC(5,4)	FAIS disclosed commission %
schedule	JSONB	Structured policy schedule data

Audit Log (INSERT-only, Partitioned by Month)

Column	Type	Notes
id	UUID PK	gen_random_uuid()
tenant_id	UUID	
actor_id	UUID	The user who performed the action
actor_type	ENUM	user / system / integration / ai
action	VARCHAR(100)	e.g. client.update, policy.inception, claim.create
resource_type	VARCHAR(50)	Table name of the affected record
resource_id	UUID	PK of the affected record

Column	Type	Notes
before_state	JSONB	<i>Snapshot before change</i>
after_state	JSONB	<i>Snapshot after change</i>
ip_address	INET	
created_at	TIMESTAMPTZ	<i>Partition key — RANGE partitioned by month</i>

```
-- No UPDATE or DELETE permitted on audit_log:
CREATE OR REPLACE FUNCTION prevent_audit_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Audit log is immutable. TG_OP=% is not permitted.', TG_OP;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER audit_immutable
BEFORE UPDATE OR DELETE ON audit_log
FOR EACH ROW EXECUTE FUNCTION prevent_audit_modification();
```


Two Independent AI Systems

System A — Recommendation RAG

Where: Admin Portal — quote recommendation screen

Who triggers it: Client presses "I have a question" on a specific recommended insurer

Data source: Insurer's product document corpus (policy wording, benefits guide, claims guide)

Context: Scoped to one insurer. No cross-insurer mixing in one chat session.

System B — Client AI Assistant

Where: Client Portal — Insights tab + contextual inline triggers

Who triggers it: Client asks any question in their own app

Data source: Their own structured data (policies, claims, financials) + insurer documents

Context: Full client financial context. Cross-policy. Natural language.

RAG Architecture — Two-Stage Retrieval

USER QUESTION: "How much am I paying for Nomsa's cover?"

STAGE 1 — QUESTION ROUTING:

Classifier identifies question type:

- "financial_data" — needs structured DB query
- "policy_question" — needs vector search in documents
- "hybrid" — needs both

STAGE 2A — DB QUERY (for financial_data or hybrid):

Backend extracts intent → generates parameterised SQL

Example: `SELECT SUM(premium_amount) FROM policies
WHERE client_id = ? AND dependant_id = ? (Nomsa)`

Result: R 184/month — fully structured, no hallucination possible

STAGE 2B — VECTOR SEARCH (for policy_question or hybrid):

Query embedded → search Pinecone / pgvector index

Returns top-K chunks from relevant policy documents

Each chunk tagged with: document_id, section, page, score

CONTEXT ASSEMBLY:

Anonymised client context + DB results + retrieved chunks

- Sent to LLM for response generation
- LLM returns JSON response (not raw text)
- Backend stores full conversation with chunk citations

PII PROTECTION:

Never sent to LLM: full name, ID number, bank account, full address, policy number, beneficiary details

The LLM receives: [CLIENT], [SPOUSE], [POLICY_A], [POLICY_B]

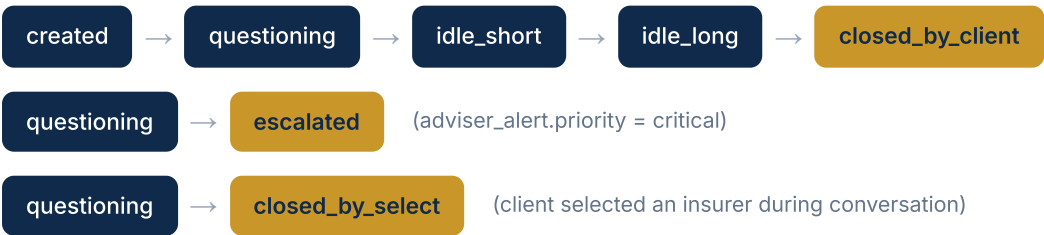
The backend maps these placeholders back to real values before displaying to the client

JSON Styling System — AI Response Format

The AI never returns plain text. Every response is structured JSON that the UI renderer interprets to build a visual card with appropriate icon, colour, and components. This ensures every response looks consistent, accessible, and branded — regardless of what the AI says.

```
{
  "response_type": "gap_detected | confirmed | not_covered | partial | warning | process |
                    definition | comparison | escalate | checklist | financial_summary |
                    goal_progress | payment_history | action_needed | insight | cover_summary",
  "tone": "reassuring | cautionary | urgent | informational | friendly | honest",
  "icon": "shield-check | shield-x | shield-alert | alert-triangle | bar-chart-2 |
          trending-up | trending-down | target | banknote | piggy-bank | activity",
  "color_scheme": "green | red | amber | blue | purple | neutral | slate",
  "emphasis_level": "low | medium | high | critical",
  "header": "Your Santam Motor policy covers this incident",
  "components": [
    { "type": "paragraph", "content": "... " },
    { "type": "callout", "content": "...", "style": "warning", "icon": "alert-triangle" },
    { "type": "metric", "label": "Monthly premium", "value": "R 1,365", "icon": "banknote" },
    { "type": "table", "headers": ["Cover type", "Limit", "Excess"],
      "rows": [ ["Comprehensive", "Market value", "R 5,000"] ] },
    { "type": "progress_bar", "label": "Emma University Fund", "value": 36, "max": 100 },
    { "type": "badge_row", "items": [{ "label": "In force", "style": "green" } ] }
  ],
  "source_citations": [
    { "document_name": "Santam Policy Wording 2024",
      "section": "Section 4.2 – Accidental damage",
      "chunk_id": "chunk_abc123", "relevance_score": 0.94, "display": true }
  ],
  "actions": [
    { "type": "contact_adviser", "label": "Speak to my adviser", "style": "primary" },
    { "type": "open_chat", "label": "Ask another question", "style": "secondary" }
  ],
  "escalate_to_adviser": false,
  "adviser_alert": {
    "trigger": "client_asked_about_gap",
    "message": "Client is asking about credit shortfall – check if they have financing",
    "priority": "medium"
  }
}
```

Conversation State Machine



REGULATORY COMPLIANCE OF AI CONVERSATIONS

Every AI conversation is stored permanently with: full message history, source chunks cited per response, LLM model version and temperature used, conversation state at each turn, and client/adviser identifiers. This record is retrievable for FAIS regulatory audit. The AI is explicitly prohibited from giving

advice — it answers questions about what a policy says or what data shows. Advice decisions remain with the human adviser.

The Problem It Solves

Each insurer has a different integration method — some have REST APIs, some use SOAP/XML, some accept SFTP file drops, some still work via email. The Integration Gateway creates one internal abstraction so that the rest of the platform never knows or cares what protocol an insurer uses. Adding a new insurer means writing one provider adapter — nothing else changes.



Supported Insurers — Initial Integration List

Insurer	Integration Method	Products	Status
Santam	REST API	Motor, Home, Business	Priority 1
Old Mutual Insure	SOAP/XML	Motor, Home, Life	Priority 1
Discovery Insure	REST API	Motor (Vitality Drive), Home	Priority 1
OUTsurance	REST API	Motor, Home, Life (OUTbonus)	Priority 1
Hollard	REST API	Motor, Funeral, Business	Priority 2
Sanlam	SFTP + REST	Life, Funeral, Investments	Priority 2

Insurer	Integration Method	Products	Status
Momentum	REST API	Life, Health, Motor	Priority 2
Allan Gray	REST API	Investments, Retirement	Priority 3
King Price	REST API	Motor (decreasing premium)	Priority 3
Clientele	SFTP / Email	Funeral, Life	Priority 3

Message Reliability — Queue & Retry

Happy Path

1. Request placed in SQS queue
2. Worker picks up & sends to insurer
3. 200 OK received
4. Response normalised & stored
5. Admin notified — quote arrived
6. Integration audit log entry created

Failure Path

1. Request placed in SQS queue
2. Worker picks up & sends to insurer
3. Error or timeout received
4. Exponential backoff retry (1m, 5m, 30m)
5. After 3 failures → dead-letter queue
6. Super admin alerted. Manual intervention flow opens.

Canonical Data Model — Quote Request

```
{
  "request_id": "uuid",
  "tenant_id": "uuid",
  "client_id": "uuid",
  "application_id": "uuid",
  "product_type": "motor",
  "insured_items": [
    {
      "type": "vehicle",
      "make": "Toyota", "model": "Corolla Quest", "year": 2022,
      "registration": "GP 123-456", "vin": "...",
      "cover_type": "comprehensive",
      "regular_driver": { "age": 34, "licence_years": 10 },
      "address": { "suburb": "Sandton", "province": "Gauteng", "postal_code": "2196" }
    }
  ],
  "client_profile": {
    "age": 34, "gender": "male", "marital_status": "married",
    "claims_last_3_years": 0, "risk_profile": "low"
  },
  "requested_cover": {
    "excess": 5000, "sum_insured": "market_value",
    "endorsements": ["car_hire", "key_replacement"]
  },
  "metadata": { "requested_at": "2026-09-05T10:30:00Z", "adviser_id": "uuid" }
}
```

Notification Channels

Channel	Primary Use	Trigger
WhatsApp Business API	Client status updates, urgent alerts	Policy inception, claim update, renewal due, payment failed
Email	Documents, detailed updates, ROA delivery	Welcome, quote ready, policy schedule, claim settlement
SMS	One-time codes, critical alerts	MFA OTP, payment failure, claim approved
In-App Push	Real-time notifications in client portal	All events while client is logged in
In-App Dashboard	Persistent action board for staff	SLA breaches, overdue tasks, integration health alerts

In-App Search Engine

Universal search is available from every screen via **Cmd+K / Ctrl+K** or the search icon in the top bar. Results are categorised into three types:

Navigation Results

Take the user to any screen or section of the app. Indexed locally — returns in under 50ms. No server call needed.

e.g. typing "claims" → shows Claims section link

Data Results

Find specific records — policies, clients (admin), claims, documents, goals. Fuzzy matched. Handles typos and partial IDs.

e.g. admin types "dlamni" → finds "Dlamini"

Action Results

Trigger actions directly from search without navigating. e.g. "report accident", "upload document", "contact adviser".

e.g. typing "bank" → shows "Update bank details" action

Search Target	Speed Target	Source
Navigation & quick actions	< 50ms	Local index (never hits server)
Own data (client portal)	< 200ms	Local cache + DB
Admin cross-client search	< 500ms	Server full-text search (pg_trgm)
Super admin global search	< 800ms	Server search across all branches

Accessibility System

Accessibility settings are stored server-side per user account and sync across all devices on login. The application is built WCAG 2.1 AA compliant as a baseline — settings are adjustments, not patches.

Language Support — 13 Languages

- English (en-ZA), Afrikaans, isiZulu, isiXhosa
- Sepedi, Sesotho, Setswana, siSwati
- Tshivenda, Xitsonga, isiNdebele
- Português, Français (global expansion)
- AI assistant responds in selected language
- System detects OS language on first launch

Screen Reader Support

- Full ARIA roles, labels, live regions
- Skip links on every page
- Focus trap in modals
- Announcements for dynamic content changes
- Charts have text-alternative tables
- Compatible: VoiceOver, TalkBack, NVDA, JAWS

Colour Blind Modes

- Deuteranopia safe (most common — ~6% of males)
- Protanopia safe
- Tritanopia safe
- Achromatopsia (complete — no colour)
- All statuses use icon + colour (never colour alone)
- Charts use patterns + colour

Additional Settings

- Text size: Small → Maximum (5 steps)
- Font: System / Inter / Atkinson Hyperlegible / OpenDyslexic
- Reduced motion (respects OS setting)
- High contrast mode
- Touch target size (up to 80×80px for motor issues)
- Extended session timeout (2 hours for form completion)

COLOUR TOKEN SYSTEM — ALL AI JSON RESPONSES ARE COLOUR-MODE AWARE

The AI response JSON uses semantic colour tokens (`"color_scheme": "red"`), never hex values. The UI renderer maps each token to the correct actual colour for the current user's colour blind mode. A user with deuteranopia sees a different actual colour for "red" — but the semantic meaning (danger/error) is always conveyed by both colour AND an icon, so no information is ever lost for any user.

Cloud Infrastructure — AWS af-south-1 (Cape Town)

Primary region: `af-south-1` (Cape Town) — mandated by POPIA data residency requirements for South African personal information. All client PII, policy data, and audit logs remain in South Africa.



Technology Stack — Cloudflare-Native Architecture

ARCHITECTURE PHILOSOPHY

Cloudflare Workers handles all edge concerns (routing, auth, rate limiting, caching, file storage, queues). **Node.js + Hono** (deployed on Workers) serves the business logic API endpoints. **FastAPI (Python)** handles AI/RAG processing, document parsing, and data transformation — deployed as a Worker service or container. **React + TypeScript** on all four portals, served from Cloudflare Pages with edge-side rendering.

Cloudflare Platform

- **Workers** — Edge API + auth middleware
- **Pages** — All 4 portals (SSR at edge)

Node.js API (on Workers)

- **Hono** — Ultra-fast Workers-native router
- **TypeScript 5** — Full type safety

FastAPI (Python — AI Services)

- **FastAPI** — Async Python API framework

- **KV** — Sessions, feature flags, cache
- **R2** — Documents, files, policy schedules
- **D1** — Edge-cached read replicas
- **Queues** — Integration gateway jobs
- **Hyperdrive** — PostgreSQL connection pooling
- **AI Gateway** — Unified LLM access + observability
- **WAF + Zero Trust** — Admin portal security

- **Drizzle ORM** — Works with D1 + PostgreSQL
- **Zod** — Schema validation
- **Jose** — JWT creation and verification
- **Resend** — Transactional email
- **Twilio SDK** — SMS
- **ws** — WebSocket (real-time)

- **LangChain** — RAG pipeline orchestration
- **OpenAI SDK** — GPT-4o + embeddings
- **pgvector** — Vector similarity search
- **PyMuPDF** — PDF document parsing
- **Pydantic v2** — Data validation + serialisation
- **Celery + Redis** — Async document indexing
- **httpx** — Async HTTP client

Compliance Summary

Regulation	Key Requirement	How Platform Addresses It
POPIA	Consent management, data residency, right to erasure, breach notification	Consent recorded per data type with timestamp. All data in af-south-1. Pseudonymisation on erasure (never hard delete). Breach alert workflow to Information Regulator within 72hrs.
FAIS	Record of Advice, commission disclosure, 5-year retention, FNA	ROA builder — immutable once signed by client (e-signature). Commission disclosed on quote comparison screen and stored. Soft delete with 5-year minimum retention enforced at DB level. FNA wizard built into onboarding.
FICA	KYC, PEP/sanctions screening, risk scoring, SAR	KYC record per client with expiry tracking. PEP/sanctions check via approved provider (GBG / Refinitiv). Risk score stored. Suspicious Activity Report workflow in admin portal.
GDPR	Data subject rights, DPA, cross-border transfers	Tenant-level data region config. Standard Contractual Clauses for any cross-border transfers. Same erasure mechanism as POPIA.

Implementation Roadmap

Phase	Duration	Deliverables
Phase 1 — Foundation	Months 1–3	Multi-tenant schema, RLS, IAM, authentication (magic link + MFA), client portal shell, admin portal shell, basic client CRM, document upload, audit log
Phase 2 — Core Operations	Months 4–6	Application wizard, policy management, claims, task management, notification system (email + WhatsApp + SMS), integration gateway (Priority 1 insurers), ROA builder
Phase 3 — Intelligence	Months 7–9	Recommendation engine, RAG system (insurer documents), client AI assistant, JSON styling system, in-app search engine, insurer onboarding portal
Phase 4 — Growth	Months 10–12	Financial goals module, market benchmarking, accessibility full suite (13 languages, colour blind modes), super admin analytics, multi-tenant licensing, Priority 2 & 3 insurer integrations

ROYAL SQUARE FINANCIAL — TENANT #1

Royal Square Financial uses the platform first, providing real-world validation of every feature before it is offered to other brokers. The super admin portal gives Royal Square leadership a command centre for the entire business. As the platform proves its value, it becomes a licensed SaaS product for other independent brokers in South Africa — creating a second revenue stream alongside the brokerage business itself.

ROYALSYNC — FULL ARCHITECTURE OVERVIEW



React Frontend — npm Package Manifest

Core Framework & Build

Package	Version	Purpose
react + react-dom	^18.3	UI library
typescript	^5.5	Type safety across all 4 portals

Package	Version	Purpose
<code>vite</code>	<code>^5.4</code>	Build tool + dev server (Cloudflare Pages compatible)
<code>@cloudflare/vite-plugin</code>	<code>latest</code>	Vite plugin for Workers + D1 + KV local dev
<code>wrangler</code>	<code>^3.x</code>	Cloudflare deployment CLI

Routing & Navigation

Package	Version	Purpose
<code>@tanstack/react-router</code>	<code>^1.x</code>	Type-safe client-side routing with file-based routes
<code>@tanstack/router-vite-plugin</code>	<code>^1.x</code>	Auto-generates typed route tree from file structure

Styling

Package	Version	Purpose
<code>tailwindcss</code>	<code>^4.0</code>	Utility-first CSS — v4 with CSS-first config (no <code>tailwind.config.js</code>)
<code>@tailwindcss/vite</code>	<code>^4.0</code>	Vite integration for Tailwind v4
<code>class-variance-authority</code>	<code>^0.7</code>	Type-safe component variant system (used by shadcn)
<code>clsx</code>	<code>^2.x</code>	Conditional class name merging
<code>tailwind-merge</code>	<code>^2.x</code>	Merges Tailwind classes without conflicts

Component Library (shadcn/ui + Radix primitives)

Package	Purpose
<code>@radix-ui/react-dialog</code>	Accessible modal dialogs (focus trap, ARIA)
<code>@radix-ui/react-dropdown-menu</code>	Dropdown menus with keyboard navigation
<code>@radix-ui/react-select</code>	Accessible select component
<code>@radix-ui/react-tabs</code>	Tab navigation (all portals)
<code>@radix-ui/react-tooltip</code>	Accessible tooltips
<code>@radix-ui/react-popover</code>	Popover panels
<code>@radix-ui/react-switch</code>	Toggle switches (accessibility settings)
<code>@radix-ui/react-checkbox</code>	Checkboxes (permission matrix)
<code>@radix-ui/react-accordion</code>	Expandable sections (policy details)
<code>@radix-ui/react-toast</code>	Toast notifications
<code>@radix-ui/react-avatar</code>	Client and adviser avatars
<code>@radix-ui/react-progress</code>	Progress bars (goals, uploads)
<code>@radix-ui/react-slider</code>	Sliders (accessibility settings, coverage amounts)

Package	Purpose
@radix-ui/react-separator	Dividers
@radix-ui/react-label	Accessible form labels

Icons

Package	Purpose
lucide-react	Primary icon library — 1,400+ icons, all used in AI JSON responses (shield-check, shield-x, alert-triangle, bar-chart-2, trending-up, banknote, etc.)
@heroicons/react	Supplement for any gaps — Heroicons 24px solid + outline sets
react-country-flag	Flag emoji components for language selector

Data Fetching & Server State

Package	Purpose
@tanstack/react-query	Server state management, caching, background refetch, optimistic updates
@tanstack/react-query-devtools	Dev tools for inspecting query cache
ky	Fetch-based HTTP client — tiny, works in Workers context

Forms & Validation

Package	Purpose
react-hook-form	Performant forms — minimal re-renders, wizard step support
@hookform/resolvers	Zod integration with React Hook Form
zod	Schema validation — shared between frontend and Hono backend

Tables, Data Grids & Virtualisation

Package	Purpose
@tanstack/react-table	Headless table engine — sorting, filtering, pagination, column visibility (client list, policy portfolio)
@tanstack/react-virtual	Virtual scrolling for large lists (1,000+ clients in admin)

Charts & Data Visualisation

Package	Purpose
recharts	Composable chart library — net worth over time, premium breakdown, goal progress

Package	Purpose
@tremor/react	Pre-built dashboard components — metric cards, bar charts, area charts (super admin analytics)

Search, Command Palette & Combobox

Package	Purpose
cmdk	Cmd+K command palette — powers the in-app universal search across all portals
fuse.js	Fuzzy search for local navigation index (offline-capable, <50ms)

File Upload

Package	Purpose
tus-js-client	Resumable chunked uploads to Cloudflare R2 — network drop safe
react-dropzone	Drag-and-drop file input component with ARIA support

Rich Text & Document Editing

Package	Purpose
@tiptap/react	Rich text editor for ROA builder, email templates, notes
@tiptap/starter-kit	Common Tiptap extensions bundle
@react-pdf/renderer	Generate ROA PDFs, policy schedules, commission statements in-browser

Dates, Numbers & Formatting

Package	Purpose
date-fns	Date manipulation — renewal calculations, KYC expiry, SLA deadlines
date-fns-tz	Timezone support (Africa/Johannesburg primary)
numeral	South African number formatting (R 1 240,00 with space thousands)

Animation & Motion

Package	Purpose
framer-motion	Declarative animations — respects prefers-reduced-motion natively. Used for page transitions, notification slide-ins, AI response reveal

Notifications & Toast

Package	Purpose
sonner	Toast notifications — dismissable, accessible, supports promise toasts for async operations

Internationalisation (i18n)

Package	Purpose
<code>i18next</code>	Core i18n framework — 13 language namespaces
<code>react-i18next</code>	React bindings — <code>useTranslation()</code> hook in every component
<code>i18next-browser-languagedetector</code>	Detects OS/browser language on first load
<code>i18next-http-backend</code>	Lazy-loads translation files from Cloudflare R2

Digital Signatures & Consent

Package	Purpose
<code>react-signature-canvas</code>	Client-side signature capture (ROA signing, mandate authorisation)
<code>@docuseal/react</code>	Enterprise e-signature flows for complex documents

Drag & Drop

Package	Purpose
<code>@dnd-kit/core</code>	Accessible drag-and-drop — Kanban task board, document reordering
<code>@dnd-kit/sortable</code>	Sortable list extension for dnd-kit

PWA & Offline (Client Portal)

Package	Purpose
<code>vite-plugin-pwa</code>	Service worker generation, offline caching strategy, Web App Manifest
<code>workbox-window</code>	Workbox runtime strategies — cache-first for static, network-first for API

Testing

Package	Purpose
<code>vitest</code>	Unit and integration testing — Vite-native, fast
<code>@testing-library/react</code>	Component testing with accessibility-first queries
<code>playwright</code>	End-to-end tests — all critical user flows across all 4 portals
<code>@cloudflare/vitest-pool-workers</code>	Run Vitest tests inside the Workers runtime for accurate edge testing

Developer Experience

Package	Purpose
<code>eslint</code> + <code>@typescript-eslint/eslint-plugin</code>	Linting with TypeScript-aware rules
<code>prettier</code>	Code formatting — consistent across all workspaces
<code>husky</code> + <code>lint-staged</code>	Pre-commit hooks — lint + type check before every commit

Package	Purpose
turbo	Monorepo build orchestration — shared packages, incremental builds

Hono API (Node.js on Cloudflare Workers)

Package	Purpose
hono	Ultra-fast framework built for Cloudflare Workers — 12x faster than Express in edge environment
@hono/zod-validator	Request body/query validation using shared Zod schemas
@hono/jwt	JWT middleware for auth on every protected route
drizzle-orm	TypeScript ORM — works with PostgreSQL (via Hyperdrive) and D1
drizzle-kit	Schema migration generation and management
@neondatabase/serverless	Neon PostgreSQL serverless driver — compatible with Cloudflare Workers
jose	JWT signing and verification (Web Crypto API — works in Workers)
resend	Transactional email via Resend API (1-line setup, React Email templates)
react-email	Build email templates in React — rendered server-side and sent via Resend
otplib	TOTP generation for MFA — works in Workers WebCrypto
qrcode	Generate MFA QR codes for authenticator app setup

FastAPI Python Service — pip Requirements

Package	Purpose
fastapi	Async Python web framework — AI and processing endpoints
uvicorn[standard]	ASGI server for FastAPI
pydantic[v2]	Data validation — matches Zod schemas on the Node.js side
langchain	RAG pipeline orchestration — document loaders, text splitters, retrievers
langchain-openai	OpenAI integration with LangChain (GPT-4o, text-embedding-3-large)
langchain-community	pgvector integration, document loaders
openai	OpenAI Python SDK — direct access for fine-grained control
pgvector	PostgreSQL vector type support for similarity search
pymupdf	PDF parsing — extract text, tables, and structure from insurer documents
python-multipart	File upload handling in FastAPI
httpx	Async HTTP client — calling external insurer REST/SOAP APIs
celery[redis]	Background tasks — document indexing, batch embedding generation

Package	Purpose
redis	Celery broker + result backend + rate limiting
boto3	AWS SDK — S3-compatible calls to Cloudflare R2
zeep	SOAP client — for SOAP/XML insurer integrations (Old Mutual, etc.)
paramiko	SFTP client — for file-based insurer integrations

Monorepo Structure

```
royalsync/
├─ apps/
│  ├─ client-portal/      # React + Vite – Cloudflare Pages
│  ├─ admin-portal/      # React + Vite – Cloudflare Pages
│  ├─ super-admin/       # React + Vite – Cloudflare Pages + Zero Trust
│  ├─ insurer-portal/    # React + Vite – Cloudflare Pages
│  ├─ api/               # Hono on Cloudflare Workers (Node.js/TS)
│  └─ ai-service/        # FastAPI (Python) – document processing + RAG
├─ packages/
│  ├─ ui/                # Shared component library (shadcn + custom)
│  ├─ schemas/           # Shared Zod schemas (frontend + API)
│  ├─ types/             # Shared TypeScript types
│  ├─ db/                # Drizzle schema + migrations
│  ├─ email-templates/   # React Email templates
│  └─ i18n/              # Translation files (13 languages)
├─ workers/
│  ├─ auth-worker/       # Edge auth middleware (JWT validation, session)
│  ├─ upload-worker/     # Chunked upload handler → R2
│  ├─ notification-worker/# Queue consumer → WhatsApp/Email/SMS dispatch
│  └─ integration-worker/# Queue consumer → Insurer API calls
├─ turbo.json            # Turborepo pipeline config
├─ pnpm-workspace.yaml   # pnpm monorepo config
└─ wrangler.toml        # Cloudflare Workers config
```

Cloudflare Workers — What Each Worker Does

Worker	Bindings Used	Responsibility
auth-worker	KV (sessions), D1 (user lookup cache)	Validates JWT on every request. Sets tenant context. Blocks unauthenticated requests at edge before they reach the API — zero cost on origin.
api-worker (Hono)	KV, Hyperdrive (→PostgreSQL), D1, R2, Queues, AI Gateway	All business logic endpoints. Routes requests to appropriate handlers. Returns JSON. WebSocket upgrade for real-time.
upload-worker	R2 (storage), KV (upload state)	Handles chunked file uploads directly to R2. Tracks upload progress in KV. Supports resume on network drop. Validates file type and size.
notification-worker	Queues (consumer), KV (dedup)	Consumes notification jobs from Cloudflare Queues. Routes to WhatsApp API / Resend (email) / Twilio (SMS). Dedup via KV to prevent double-send.

Worker	Bindings Used	Responsibility
integration-worker	Queues (consumer), KV (state), R2 (docs)	Consumes insurer integration jobs. Calls FastAPI service for insurer-specific normalisation. Handles retry and dead-letter. Stores responses in R2.
search-worker	Hyperdrive, D1 (index cache)	Handles admin full-text search across clients, policies, claims. Uses pg_trgm on PostgreSQL. Caches common queries in D1.

ROYAL SQUARE FINANCIAL (PTY) LTD

Platform Architecture & Technical Planning Document · Version 1.0 · September 2026

CONFIDENTIAL — For internal and approved partner use only